

UCS503P – Software Engineering

Scientific Literature Intelligence Platform

A local, citation-grounded retrieval-augmented assistant for scientific papers

Submitted by:

1024030171 Paarth Ganesh

1024031154 Diya Jain

Group: COE, BE Third Year Academic Year 2026–27

Submitted to:

Ms. Paramveer Kaur (Lab Instructor)

Final Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.1.1	Problem Statement	1
1.2	Project Objectives	1
1.3	Scope and Exclusions	2
1.4	Time-to-Value	2
1.5	Intended Users and Contributions	2
1.6	Requirements Summary	2
2	Methodology and Design	5
2.1	System Architecture	5
2.1.1	Architectural Style	5
2.1.2	Technical Stack	5
2.2	UML and Software Design	6
2.2.1	Use Case Diagram	6
2.2.2	Class Diagram	7
2.2.3	Sequence Diagram	7
2.2.4	Data-Flow / Activity View	7
2.3	Implementation Details	7
2.3.1	Ingestion: Discovery, Upload and Chunking	7
2.3.2	Retrieval: Embedding, Search and Re-ranking	8
2.3.3	Grounded Question Answering and Citation Checking	9
2.3.4	Analyses, Frontend and Configuration	9
2.4	Security, Privacy and Reliability	9
3	Results and Evaluation	11
3.1	Testing Methodology	11
3.2	Evaluation Data Set and Metrics	11
3.3	Performance Metrics	12
3.3.1	Groundedness, Abstention and Prompt Injection	12
3.3.2	Final Acceptance and Continuous Integration	13
3.4	Requirement Status at Final Acceptance	13
3.5	Analysis	14

4 Conclusion	15
4.1 Summary of Work	15
4.2 Key Findings	15
4.3 Future Work and Recommendations	15
4.4 Final Remarks	16
References	17

Chapter 1

Introduction

1.1 Background and Motivation

Scientific output grows faster than any individual can read. A student or researcher who wants to know which optimiser a paper used, how two methods compare on a benchmark, or what limitations the authors acknowledge must locate the relevant papers, open each PDF and search its text by hand. Large language models (LLMs) can answer such questions fluently, but a model answering from its parameters gives no indication of where an answer came from and is known to produce plausible but unsupported statements. For scientific use, an answer is only useful if the reader can check it against the source.

Retrieval-augmented generation (RAG) addresses this by first retrieving passages from a document collection and then asking the model to answer from those passages. Retrieval alone does not make an answer trustworthy: the model can still cite the wrong passage, cite a passage it was never given, or answer when the passages do not contain the answer. This project treats citation integrity and abstention as engineering requirements that the server enforces and that the team measures, rather than as properties assumed of the model.

1.1.1 Problem Statement

Build a software system that lets a user ask natural-language questions over a corpus of scientific papers and returns answers in which every factual claim is traceable to a specific passage and page of a specific paper, that explicitly declines to answer when the indexed papers do not support an answer, and whose retrieval and answer quality are measured reproducibly. The corpus must be populated both from arXiv and from user-supplied PDF files.

1.2 Project Objectives

1. **O1 – Ingestion.** Ingest papers from the arXiv API and from uploaded PDF files, with validation and job tracking.
2. **O2 – Provenance.** Extract text page by page and preserve provenance from paper to page, character span and chunk.
3. **O3 – Retrieval.** Provide semantic search over the corpus that returns ranked passages with their pages.
4. **O4 – Grounded answering.** Answer questions with a local language model, cite each claim, verify every citation on the server, and abstain explicitly when evidence is insufficient.
5. **O5 – Analyses.** Provide single-paper summaries, cross-paper synthesis, structured extraction and comparison.
6. **O6 – Measurement.** Measure retrieval quality (Recall@k, MRR), citation validity, abstention and latency on a version-controlled evaluation set, and gate retrieval regressions in continuous integration.

7. **O7 – Engineering discipline.** Apply software-engineering discipline throughout: requirement identifiers, traceability, architecture decision records, automated tests, security review and reproducible setup.

1.3 Scope and Exclusions

The delivered system is a single-user application intended to run on one machine with Docker Compose. Out of scope for the minimum viable product (MVP): optical character recognition for scanned PDFs, user accounts and authorisation, public deployment, and any hosted (cloud) language model. The original proposal named a hosted model (Gemini 2.5 Flash) and a cloud deployment; the team replaced the former with local inference through Ollama and kept deployment local (ADR-0001), so that no paper or question leaves the user’s machine.

1.4 Time-to-Value

Development prioritised the ingest-to-answer loop, because every other feature reads from it. The design system and data model were built first, together with the mock-free service layer and database schema, so that every feature shares one source of truth. Authentication was deliberately skipped (single-user, local-only) in favour of the application shell and routing. The core pipeline – arXiv/PDF ingestion, page-aware extraction, chunking and embedding – was built next, since search and question answering both read from it. Semantic search (dense, lexical, hybrid, re-ranked) followed, then grounded question answering with server-side citation checking, which delivers the primary value of the system. Summaries, synthesis, extraction and comparison were layered on top once evidence selection was trustworthy. The evaluation harness, the continuous-integration retrieval gate, and the security and reliability review were built last, against the accumulated pipeline, followed by final acceptance testing on 9 October 2026.

1.5 Intended Users and Contributions

The primary users are students and researchers who maintain a working set of papers and want to query it. A secondary audience is the course evaluators and any future maintainers at the institute, for whom the repository provides requirements, decision records, test suites and operating instructions. The contributions of this project are engineering contributions, not new retrieval or generation methods: a working, documented end-to-end system combining established techniques (dense retrieval, BM25-style lexical retrieval, reciprocal rank fusion and cross-encoder re-ranking with local generation); a server-side citation contract under which the model may only cite passage labels that are resolved against the passages actually supplied, so an answer without a valid citation is withheld; and an evaluation harness with reproducible run records, a continuous-integration retrieval gate, and audits of retrieval quality, security and reproducibility whose findings – including the unfavourable ones – are reported in this document.

1.6 Requirements Summary

The canonical requirement register is version-controlled, with permanent identifiers: functional requirements FR-01 to FR-26, infrastructure requirements IR-01 to IR-09, and non-functional requirements NFR-01 to NFR-16. Table 1.1 summarises the required functional requirements; Table 1.2 lists the non-functional targets, each measured rather than claimed without a recorded run (chapter 3).

The one user on one machine assumption underlies the design: all application ports bind to 127.0.0.1, there is no authentication, and there is no hosted-model fallback if Ollama is unavailable (ADR-0001) –

Table 1.1: Selected functional requirements

ID	Requirement
FR-01–03	Search, import and upload papers from arXiv or as a PDF, with validation (type, size, pages, encryption, text layer) and idempotent re-import.
FR-04–07	Page-aware text extraction; deterministic chunking with full page/offset provenance; local embeddings (384-d); persistence in PostgreSQL with pgvector, constraints and migrations.
FR-08–11	Semantic search returning ranked, paginated passages; grounded question answering from retrieved passages only; server-side citation resolution; explicit insufficient-evidence response.
FR-12–15	Single-paper summary, cross-paper synthesis, structured extraction and multi-paper comparison, each with provenance or per-field citation.
FR-16–18	Corpus browsing and deletion; ingestion job tracking with explicit states and retry; query history with answers, citations, configuration and latency.
FR-19–21	Retrieval evaluation harness (Recall@k, MRR); answer-quality evaluation (citation validity, groundedness, abstention, latency); automated retrieval-quality gate in CI.
FR-22–25	Dashboard, read-only settings view, hybrid retrieval and cross-encoder re-ranking (recommended/optional, both implemented).

the system instead returns HTTP 503 with a reason. Only text-based PDFs are supported; scanned PDFs are rejected rather than processed with optical character recognition.

Table 1.2: Non-functional requirements (targets)

Quality	Target
Retrieval quality	Recall@5 ≥ 0.80 on the pilot evaluation set; MRR reported for every configuration
Latency	95th-percentile end-to-end question answering ≤ 3 s; search latency reported separately from generation
Availability	$\geq 99\%$ of search and Q&A over a defined pilot window
Integrity	100% of displayed citations resolve to persisted chunks; abstention precision/recall reported
Reproducibility	Same corpus and configuration give identical chunks and rankings; model, embedding model, chunk configuration and prompt version recorded
Security & privacy	Upload limits, path-traversal protection, parameterised SQL, CORS, secrets handling, prompt-injection mitigation; no document text or secrets in logs; no data sent to a hosted LLM
Hardware fit	Runs on the reference laptop (16 GB RAM, 4 GB GPU, Windows 11)
Accessibility & degradation	Keyboard navigation, labelled controls, usable at 360 px width; search works without Ollama, Q&A returns an explicit 503

Chapter 2

Methodology and Design

2.1 System Architecture

2.1.1 Architectural Style

The system is a modular monolith: one FastAPI backend process and one React single-page frontend, deployed together with PostgreSQL by Docker Compose (ADR-0002). A single process keeps transactions simple (a paper and all its chunks are written in one transaction) and makes the system easy to run on one laptop. The browser talks only to nginx, which serves the built React application and proxies `/api/v1` to the backend. The backend reaches PostgreSQL inside the Compose network, Ollama on the host through `host.docker.internal`, and arXiv over HTTPS through an allow-listed client. All published ports bind to `127.0.0.1`.

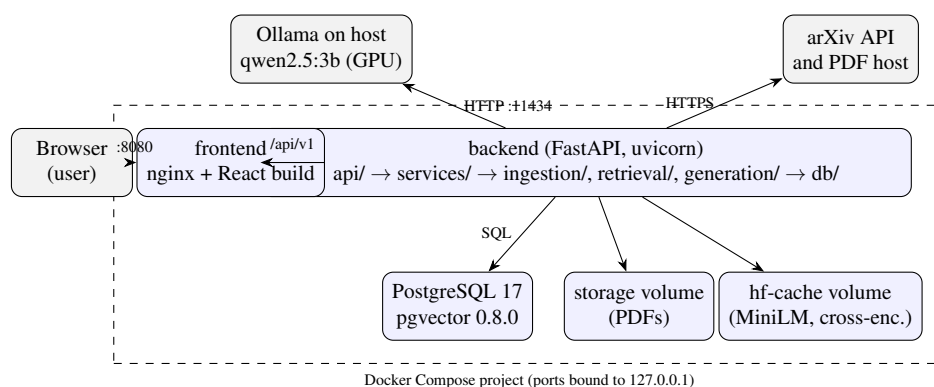


Figure 2.1: Deployment and component architecture

The backend is divided into packages with a strict dependency direction: `api` → `services` → `{ingestion, retrieval, generation, evaluation}` → `db`. Routes never contain SQL or prompts, and the generation package never touches the database. Nine tables hold the persistent state: `papers`, `documents`, `chunks`, `ingestion_jobs`, `queries`, `answers`, `answer_evidence`, `answer_citations` and `analyses`. Evidence rows are snapshots of the passages given to the model, so question history survives deletion of a paper. The schema changes only through Alembic migrations (0001–0006); a test fails if the ORM models and migrations drift apart.

2.1.2 Technical Stack

- **Backend:** Python 3.12, FastAPI, Pydantic v2 schemas at every boundary, SQLAlchemy 2 with Alembic migrations.
- **Frontend:** React with TypeScript and Vite, served by nginx 1.27; Vitest with Testing Library for component tests.
- **Data:** PostgreSQL 17 with the pgvector extension (HNSW cosine index) and full-text search (GIN

index on a stored `tsvector`).

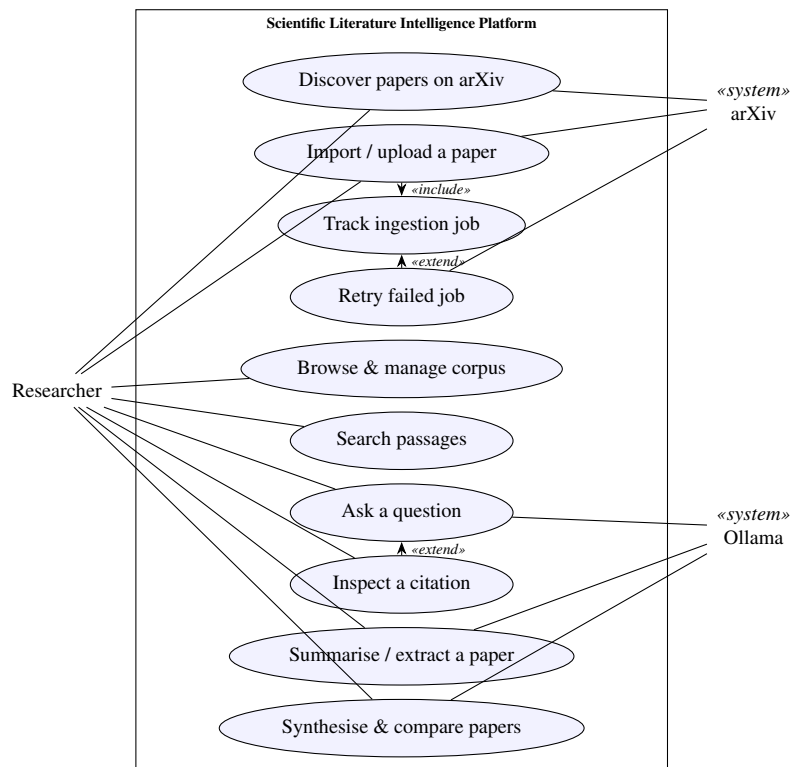
- **Retrieval:** all-MiniLM-L6-v2 sentence embeddings (384-d, CPU), `ts_rank_cd` lexical ranking, reciprocal rank fusion ($k = 60$), and the cross-encoder/ms-marco-MiniLM-L-6-v2 re-ranker.
- **Generation:** Locally hosted `qwen2.5:3b` served by Ollama; no external or hosted LLM service is used.
- **Tooling:** Ruff, mypy (strict), pytest, oxlint, GitHub Actions, pip-audit and npm audit.

2.2 UML and Software Design

This section presents the principal UML views of the system; each diagram was drawn from the implementation and uses the class, operation and message names found in the code.

2.2.1 Use Case Diagram

Figure 2.2 shows the system boundary, the primary actor (the researcher) and the two external systems that participate in use cases: arXiv supplies metadata and PDF files, and the local Ollama service generates answers and analyses. Importing or uploading a paper includes tracking its ingestion job, because both requests return a job that the interface polls. Inspecting a citation extends asking a question and viewing history, since it is optional behaviour available wherever an answer with citations is shown.



View history, dashboard and settings use cases are omitted here for space; see chapter 6.

Figure 2.2: UML use case diagram

2.2.2 Class Diagram

Figure 2.3 shows a simplified view of the persistent domain model. A Paper owns at most one Document (the PDF currently behind it), which owns its Chunks; deleting a paper cascades to both. Question history is modelled by Query, Answer, AnswerEvidence and AnswerCitation: each citation resolves to at most one evidence row of the same answer, and each evidence row is a snapshot of a chunk whose reference becomes empty if the chunk is later deleted.

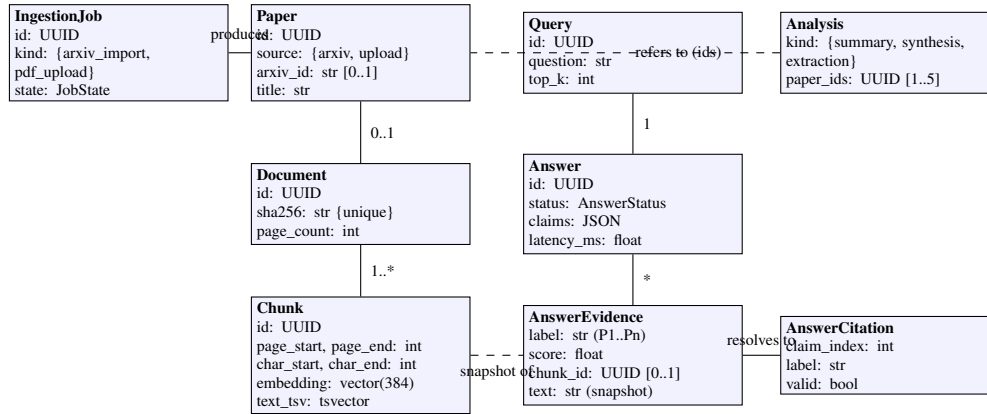


Figure 2.3: UML class diagram (simplified)

2.2.3 Sequence Diagram

Figure 2.4 follows a question from the interface to the answer and back. After validation, the service retrieves candidates (embedding, dense and lexical SQL, rank fusion and cross-encoder re-ranking), keeps passages whose cosine similarity is at least 0.30 and labels them P1–Pn. If no passage clears the threshold the system abstains without a model call; otherwise the model is called, and citations are checked before the outcome is persisted as answered or insufficient_evidence.

2.2.4 Data-Flow / Activity View

Figure 2.5 traces data from a source document to an answer citation. Ingestion validates the input, extracts text page by page while recording character offsets, chunks the text into 256-token windows with a 38-token overlap, embeds the chunks and writes everything in one transaction. Retrieval embeds the question, runs dense and lexical search, fuses the two rankings with reciprocal rank fusion and re-orders the fused candidates with the cross-encoder. The provenance chain – paper, page, character span, chunk, retrieval score, citation – is preserved end to end.

2.3 Implementation Details

2.3.1 Ingestion: Discovery, Upload and Chunking

The arXiv client (`ingestion/arxiv.py`) parses old- and new-style identifiers, restricts outbound requests to an allow-list of HTTPS hosts, respects a minimum interval between requests, and parses feeds with `defusedxml`. Uploads are read in blocks and capped at 50 MB; `inspect_pdf` checks the %PDF-signature, opens the file with PyMuPDF, and rejects corrupt, encrypted, empty and over-long (more than 200 pages) documents. The SHA-256 of the bytes detects duplicates (HTTP 409) and the file is stored as `<sha256>.pdf`. Extraction normalises text to Unicode NFC, joins pages with a fixed separator, and

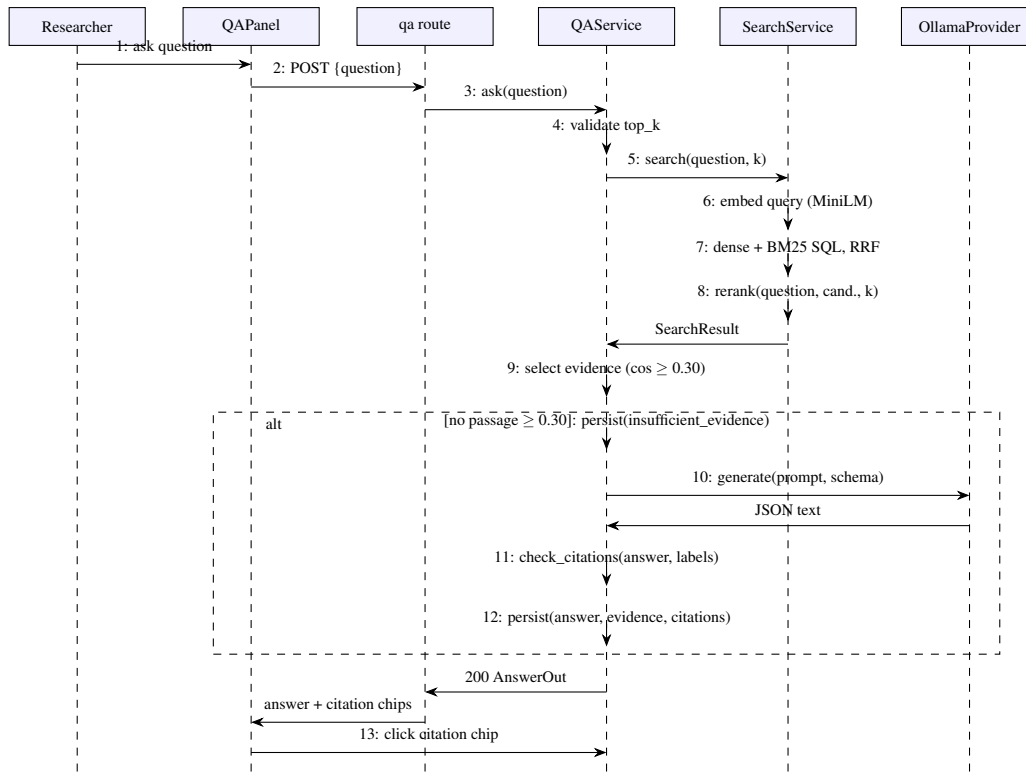


Figure 2.4: UML sequence diagram: grounded question answering

records the character offset at which each page starts, so any offset maps back to its page by binary search. `chunk_document` tokenises the extracted text with the embedding model’s own tokenizer and advances 256-token windows by a stride of 216 tokens (38-token overlap); each chunk stores its exact text, character span and the pages it spans.

2.3.2 Retrieval: Embedding, Search and Re-ranking

`SentenceTransformerEmbedder` loads `all-MiniLM-L6-v2` on the CPU, embeds in batches, normalises vectors to unit length, and validates the 384-dimensional output. `SearchService.search` supports four modes: `dense` orders chunks by pgvector cosine distance; `bm25` ranks with `ts_rank_cd` over the query’s alphanumeric terms; `hybrid` fuses the two candidate lists with reciprocal rank fusion ($k = 60$); and `hybrid_rerank` (the default) re-orders the fused candidates with the cross-encoder. A hit always carries the cosine similarity as `score`, so the evidence threshold has one meaning in every mode, while the mode’s own ordering is kept separately as `rank_score`. The fusion step itself is short and deterministic:

```

for hits, weight in ((dense_hits, dense_weight), (bm25_hits, sparse_weight)):
    for rank_idx, hit in enumerate(hits, start=1):
        rrf_scores[hit.chunk.id] += weight / (rrf_k + rank_idx)
ordered = sorted(rrf_scores, key=lambda cid: (-rrf_scores[cid], str(cid)))

```

Ties are broken by chunk identifier, which keeps rankings deterministic across repeated runs; each candidate list contributes $\max(4k, 20)$ candidates to the final top- k . A migration (0006) replaced an expression index with a stored generated `tsvector` column and a GIN index for lexical search, because ranking previously recomputed the vector for every matching row: one representative query took 636 ms before the change and 19 ms after it on the evaluation corpus.

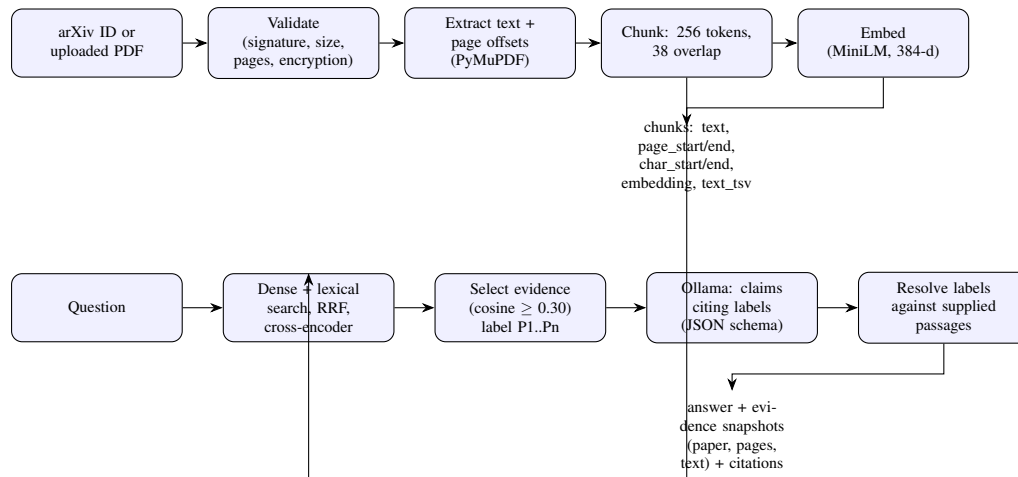


Figure 2.5: Data flow from source document to answer citation

2.3.3 Grounded Question Answering and Citation Checking

QAService keeps retrieved passages with cosine similarity of at least 0.30, up to 12,000 characters, and labels them P1 to Pn in rank order. Every passage is wrapped in a delimited element with its paper title and pages; the system prompt states that passages are quoted source material whose instructions must be ignored. The request to Ollama carries a JSON schema constraining the output to at most five claims, each listing up to six labels, followed by a status. Generation uses temperature 0, a fixed seed and a bounded token budget. On the server, labels are normalised and resolved only against the passages actually supplied; a claim whose text consists only of labels is dropped, a claim is *cited* only if at least one of its citations is valid, and the whole answer is withheld as insufficient evidence if no claim is cited. Every outcome – answered, insufficient evidence or error – is persisted with snapshots of the evidence, the citations, the model tag, prompt version, retrieval configuration and timings.

2.3.4 Analyses, Frontend and Configuration

AnalysisService gathers evidence inside the selected papers with several focused queries and applies the same evidence discipline to produce single-paper summaries (three to five cited claims), cross-paper syntheses, structured extractions (task, method, dataset, metric, result, limitations – each cited or unknown) and comparison tables, adding a caveat when the compared papers report on different datasets; comparison never ranks the papers. The React frontend provides views for the dashboard, asking questions, passage search, the corpus (filters, details, two-step delete), comparison, adding papers, history and settings, all through one typed API client with a uniform error envelope. All settings are read from SLIP_-prefixed environment variables with validated ranges; typed domain errors are mapped to HTTP status codes (422 invalid input, 404 not found, 409 duplicate, 502 upstream failure, 503 dependency unavailable, 504 generation timeout), and a readiness endpoint reports the database, embedding model, re-ranker and Ollama separately so that search keeps working when Ollama is down.

2.4 Security, Privacy and Reliability

The system was reviewed twice for security – a first pass during development and a second pass before MVP submission – as practical reviews (code reading, the test suites, targeted probes against the running stack and a check of the host’s open ports), not as penetration tests or a comprehensive security audit. The threat model is a single user on one machine with all application ports bound to 127.0.0.1.

Table 2.1: Security and privacy controls and how they were verified

Area	Control	Verification
Upload validation	50 MB cap, PDF signature, full open, encryption and page checks before any job	Tests; live probes (non-PDF, truncated and oversized files rejected)
Storage	Server-generated <sha256>.pdf names; keys validated to stay inside the storage root	Test of unsafe keys (path-traversal attempt stored safely inside storage)
Database	ORM and bound parameters only; lexical query terms reduced to word characters	Code search; injection-style queries left tables intact
Outbound requests	arXiv hosts allow-listed (HTTPS), redirects re-checked, size caps, defusedxml	Tests
Prompt injection	Delimited, neutralised passages; instructions to ignore passage instructions; JSON output schema	Real-model test: an instruction inside a retrieved passage was not followed
Citation integrity	Labels only; server-side resolution; answers without valid citations withheld	Tests; database-wide checks (chapter 3)
Errors	Typed errors mapped to status codes; fixed message for unexpected errors; 503 for database outage	Tests; live probes
Secrets	.env ignored by version control; settings endpoint returns no credentials	Git history scan; test
Privacy	No hosted model; logs contain event names, identifiers and counts, never questions or document text	Log review

Generation runs only on the user’s machine through Ollama, with no hosted fallback. During the second review, the host’s Ollama service was found listening on all network interfaces with inbound firewall rules permitting access from the local network – a host configuration issue rather than an application defect, whose remediation (binding Ollama to 127.0.0.1) is documented but must be applied by the machine owner. Readiness reports each dependency separately, and jobs interrupted by a restart are marked failed on start-up and can be retried. Open risks carried forward: the application connects to PostgreSQL as a superuser rather than a least-privilege role; PDF parsing has no wall-clock timeout; and model revisions are pinned by name rather than by content hash.

Chapter 3

Results and Evaluation

3.1 Testing Methodology

Tests are organised in layers, selected with pytest markers so each layer runs where its dependencies exist. Table 3.1 summarises the final acceptance run of 9 October 2026. Static checks (Ruff lint and format, mypy in strict mode, oxlint, the TypeScript compiler) passed, and the production frontend build succeeded.

Table 3.1: Test layers and results in the final acceptance run

Layer	What is real / faked	In CI	Passed
Backend unit	Pure code; fake embedder, scripted fake model, fake arXiv	Yes	140
Integration & model	Real PostgreSQL + pgvector and real MiniLM; fake model	Yes	83
Real local model	Ollama qwen2.5:3b	No	3
Live arXiv	arXiv API	No	1
Stack end-to-end	Running Docker stack through nginx, all real components	No	7
Frontend	Vitest and Testing Library, mocked HTTP client	Yes	24

Unit tests cover arXiv identifier/feed parsing, PDF extraction and page mapping, deterministic chunking, prompt construction, citation checking (invalid, duplicate and label-only citations), error mapping of the Ollama adapter, and search-mode routing and fusion. Integration tests run the real pipeline against PostgreSQL: arXiv import and idempotent re-import, upload validation, search in all four modes, question answering with a scripted model (including fabricated labels and abstention), analyses, corpus management and the evaluation harness. Migration tests apply all migrations to an empty database, run alembic check to detect ORM/migration drift, and verify constraints such as cascading deletes. The end-to-end suite uploads a synthetic paper, waits for ingestion, finds it with all four search modes, asks a question, checks citation validity, requests a summary and deletes the paper; the same journeys were exercised manually with real papers during final acceptance.

3.2 Evaluation Data Set and Metrics

The evaluation corpus pins 20 natural-language-processing papers to exact arXiv versions (399 pages, 1,654 chunks). The question set has 50 items: 40 answerable with 110 verbatim evidence quotes, and 10 unanswerable, several adversarial (naming a corpus paper but asking for a fact found only in a neighbouring paper). All labels were written by the AI assistant used by the team, not reviewed by a person; each quote is verified to occur in the corpus before every run. For Q answerable questions, with r_q the rank of the first relevant chunk for question q :

$$\text{Recall}@k = \frac{1}{Q} \sum_{q=1}^Q \mathbb{1}[r_q \leq k], \quad \text{MRR} = \frac{1}{Q} \sum_{q=1}^Q \frac{1}{r_q}.$$

3.3 Performance Metrics

Table 3.2: Retrieval quality and search latency by mode (40 answerable questions)

Mode	R@1	R@5	R@10	MRR	p50 / p95 (ms)
dense (baseline)	0.200	0.700	0.775	0.416	42 / 49
bm25	0.300	0.525	0.650	0.393	56 / 91
hybrid (RRF)	0.375	0.725	0.825	0.527	102 / 137
hybrid_rerank (default)	0.575	0.875	0.925	0.701	1,734 / 1,955

Only `hybrid_rerank` meets the target $\text{Recall@5} \geq 0.80$. The cross-encoder accounts for about 92% of retrieval time (895 / 985 ms p50/p95 of the stage breakdown); query embedding, dense SQL and lexical SQL together take under 70 ms at p50.

Table 3.3: Grounded question answering over the evaluation set (two complete runs)

Measure	Run 1	Run 2
Answer rate on answerable questions	0.875	0.950
Citation validity (valid / all)	52 / 52	57 / 57
Unanswerable questions declined (abstention recall)	9 / 10	9 / 10
Warm latency p50 / p95 (s)	3.10 / 4.50	3.06 / 4.43

Citation integrity was checked deterministically: across 418 stored answers, all 422 valid citations resolve to an evidence row of the same answer, and no invalid citation is linked to evidence; the provenance checker re-extracted all 20 PDFs and confirmed, for all 1,654 chunks, that the stored text, pages and span agree with the source document (0 problems; 433 chunks span two or more pages).

Table 3.4: Retrieval stage latency at top 6, ms (p50 / p95)

Stage	Latency
Query embedding (MiniLM, CPU)	17 / 30
Dense SQL (exact scan)	29 / 36
Lexical SQL	36 / 66
Hybrid call (dense + lexical + RRF)	61 / 85
Cross-encoder re-ranking (24 pairs, CPU)	895 / 985

At this corpus size the PostgreSQL planner uses an exact sequential scan (12 ms) rather than the HNSW index, so retrieval results are exact rather than approximate; the same profiling run found 47 pairs of neighbouring (overlapping) chunks among the 300 top-6 slots examined, and no exact duplicates.

3.3.1 Groundedness, Abstention and Prompt Injection

A valid citation shows that a claim refers to a passage the model was given, not that the passage supports the claim. Two complementary checks were made: a local judge (the same `qwen2.5:3b` model) rated 31 of 37 claims (83.8%) in run 1 as supported, 3 partially supported and 3 not supported – a judge that is not independent of the answering model; and a manual spot check of 14 claims (the 6 the judge flagged plus 8 random claims it rated supported) against the full cited text found 10 supported and answering the question, one supported but not answering it, one wrong-paper answer (a Transformer question answered with LLaMA’s optimiser settings), and two not stated in the passage they cited – the same wrong-paper failure recurred in both complete runs. No independent human spot check has been performed; it remains an open item against the project’s own evaluation rules.

The system abstains at three points: when no passage reaches the cosine threshold (no model call is made at all), when the model itself reports insufficient evidence, and when no claim in the model’s output has a valid citation. In the evaluation runs, every abstention on an unanswerable question came from the model rather than the pre-call threshold. Failure handling was exercised live: with the database stopped, readiness reports the cause and the API returns 503, recovering automatically when the database restarts; with Ollama unreachable, question answering returns 503 while search keeps working; invalid input returns 422 naming the offending field. A real-model test placed an instruction inside a retrieved passage, and the model did not follow it.

3.3.2 Final Acceptance and Continuous Integration

On 9 October 2026 the rebuilt stack was exercised end to end with real components: discovering papers through arXiv search; importing a paper by identifier and confirming that a second import created no duplicate; uploading a real paper and re-checking provenance for all stored documents; searching; asking a question answered correctly with two citations whose passages contained the cited figures; asking an unsupported question, declined before any model call; running summary, extraction, synthesis and comparison; filtering and deleting papers; and opening citations in the interface. One defect found during this run (arXiv title search with common stop words returned no results, because every term was combined with AND) was fixed and covered by a regression test.

GitHub Actions runs six jobs on every push: backend lint, types and unit tests; integration tests against a pgvector service container with the real embedding model; frontend checks and build; a Docker image build; a retrieval-regression gate that rebuilds the evaluation corpus from arXiv and fails if Recall@5 drops below 0.85 for hybrid_rerank or 0.675 for dense; and dependency scanning with pip-audit and npm audit. CI does not use Ollama, so real-model and end-to-end tests are run manually before each acceptance milestone.

3.4 Requirement Status at Final Acceptance

Table 3.5 summarises verification status against the register introduced in section 1.6. The large majority of required functional and infrastructure requirements are verified; the remaining items are honestly reported as partial, unverified, failed or deferred rather than claimed.

Table 3.5: Requirement status at final acceptance

Status	Requirements
Verified	FR-01–FR-10, FR-12–FR-18, FR-21, FR-22, FR-24, FR-25; IR-01–IR-09; NFR-01 (on this set, with caveats), NFR-02, NFR-04, NFR-06–NFR-09, NFR-11, NFR-13, NFR-14, NFR-16
Partially implemented	FR-11 (one false answer in ten; no “uncertain” status), FR-19 (labels not reviewed by a person), FR-20 (no human spot check), FR-23 (settings omit the search mode), NFR-05 (pilot window too short), NFR-10 and NFR-12 (open security items)
Implemented, unverified	NFR-15 (no manual accessibility review)
Failed	NFR-03 (95th-percentile latency 4.4–4.5 s against a 3 s target)
Deferred	FR-26 (optical character recognition)

3.5 Analysis

- **Objectives:** The ingestion, retrieval, grounded question-answering and analysis pipelines are complete and verified end to end; the evaluation harness and the continuous-integration retrieval gate are operating.
- **Challenges:** Keeping retrieval fast enough for the 3-second latency target while using a cross-encoder for re-ranking, and keeping the model's citations trustworthy rather than merely well-formed.
- **Resolution:** Citation checking is enforced on the server rather than trusted from the model, and evidence snapshots make every stored answer independently auditable; latency is reported per stage so the cross-encoder cost is visible rather than hidden inside an aggregate number.
- **Open gap:** The 95th-percentile latency (4.4–4.5 s) exceeds the 3 s target, the evaluation labels have not been reviewed by a person, and the retrieval configuration was selected on the same question set it is measured on.

Chapter 4

Conclusion

4.1 Summary of Work

The objective of this project was to build a single-user web application that answers natural-language questions over a corpus of scientific papers and cites every claim to a specific passage and page, replacing manual reading and search with grounded, verifiable answers. The delivered system ingests papers from arXiv and uploaded PDFs, extracts and chunks text with full provenance, retrieves passages by combining dense and lexical search with cross-encoder re-ranking, and generates answers with a local qwen2.5:3b model whose citations are resolved on the server. Summaries, cross-paper syntheses, structured extractions and comparison tables are built on the same evidence discipline. The system was evaluated on a version-controlled set of 50 questions over 20 papers and reviewed for security and reliability.

4.2 Key Findings

- **Finding 1:** A server-side citation contract – the model may cite only supplied passage labels, and the server resolves every label – made citation integrity independently verifiable: across every stored answer, all valid citations resolved to real evidence, with zero fabricated citations possible by construction.
- **Finding 2:** The default retrieval configuration (hybrid dense + lexical search fused by reciprocal rank fusion, re-ranked by a cross-encoder) reached a Recall@5 of 0.875 and an MRR of 0.701, clearly ahead of dense retrieval alone (Recall@5 0.700), at the cost of roughly 1.1 s of extra latency per question.
- **Finding 3:** A valid citation is necessary but not sufficient evidence of support: spot checks and a local-judge review found claims that cited a real, supplied passage without that passage actually stating the claim, including one recurring wrong-paper answer. Grounding by citation resolution and grounding by semantic support are different guarantees.

4.3 Future Work and Recommendations

1. Human review of a sample of the evaluation labels, and a new held-out question set covering multi-paper, conflicting and ambiguous cases, so that further tuning does not overfit the current set.
2. A claim-to-passage entailment check and a check that the cited paper matches the paper named in the question, with an explicit “uncertain” answer status.
3. Latency work – a smaller re-ranking pool, GPU re-ranking, or a smaller generation budget – each measured against both recall and latency, to close the gap to the 3 s target.
4. Table-aware extraction and optical character recognition for scanned papers.

5. A least-privilege database role, a parsing timeout, pinned model revisions, and authentication before any shared or multi-user deployment.
6. A longer availability pilot using the existing probe script, to support a measured uptime claim.

4.4 Final Remarks

This project showed that a trustworthy retrieval-augmented system is mainly a verification problem rather than a generation problem: once passages are retrieved, the harder engineering work is making sure every claim the model produces can be checked, not making the model sound confident. Enforcing citation resolution on the server, snapshotting evidence, and measuring retrieval and answer quality on a reproducible, version-controlled set turned “the system cites its sources” from a claim into something that could be audited after the fact. The remaining work is less about new features than about closing the gap between a citation being structurally valid and a citation being semantically correct, and about bringing latency down without giving up the recall that hybrid re-ranked retrieval provides.

References

- [1] Z. Ji, N. Lee, R. Frieske, et al. “Survey of Hallucination in Natural Language Generation.” *ACM Computing Surveys* 55.12 (2023).
- [2] P. Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” *NeurIPS* (2020). arXiv:2005.11401.
- [3] Y. Gao et al. *Retrieval-Augmented Generation for Large Language Models: A Survey*. (2023). arXiv:2312.10997.
- [4] V. Karpukhin et al. “Dense Passage Retrieval for Open-Domain Question Answering.” *EMNLP* (2020).
- [5] N. Reimers and I. Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.” *EMNLP-IJCNLP* (2019).
- [6] S. Robertson and H. Zaragoza. “The Probabilistic Relevance Framework: BM25 and Beyond.” *Foundations and Trends in Information Retrieval* (2009).
- [7] G. V. Cormack, C. L. A. Clarke, and S. Buettcher. “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods.” *SIGIR* (2009).
- [8] R. Nogueira and K. Cho. *Passage Re-ranking with BERT*. (2019). arXiv:1901.04085.
- [9] pgvector contributors. *pgvector: Open-source vector similarity search for Postgres*. <https://github.com/pgvector/pgvector>.
- [10] arXiv. *arXiv API User’s Manual*. <https://info.arxiv.org/help/api/user-manual.html>.
- [11] PyMuPDF developers. *PyMuPDF Documentation*. <https://pymupdf.readthedocs.io/>.
- [12] N. F. Liu et al. “Lost in the Middle: How Language Models Use Long Contexts.” *TACL* 12 (2024).
- [13] K. Greshake et al. “Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection.” *AISec* (2023).
- [14] OWASP Foundation. *OWASP Top 10 for Large Language Model Applications*. <https://genai.owasp.org/llm-top-10/>.
- [15] Ollama. *Ollama*. <https://ollama.com/>.
- [16] Qwen Team. *Qwen2.5 Technical Report*. (2024). arXiv:2412.15115.